

CDFD Runtime Paper 02: Layered Ontology for Adaptive Systems

Steve Bico Mujjabi, MD

Independent Researcher

ORCID: 0009-0001-0556-5516

May 2026

Abstract

Executive synthesis. The CDFD Runtime separates numerical mechanics from meaning. The engine evolves arrays; the ontology layer explains what those arrays represent in a domain. This paper upgrades the earlier “5-layer” account into a more explicit layered stack: runtime mechanics, meta-ontology, domain ontologies, application ontologies, and context-specific memory. It also explains how the discovery layer can propose new schemas, while keeping every new schema in candidate status until it survives validation.

Runtime evidence path

Prior CDFD mathematics $\longrightarrow$ CDFL/runtime state $\longrightarrow$ bounded output $\longrightarrow$ falsification target.

Figure 1: Conceptual schematic for the runtime papers. The engine translates the mathematical frame from the prior CDFD papers into executable CDFL/runtime diagnostics; candidate outputs remain tied to explicit tests before any stronger claim is made.

1 Prior CDFD Basis

This manuscript does not rederive the mathematical core of CDFD. It uses the Mujjabi Adaptive Operating Ratio and the constrained-vacuum notation introduced in Part I [1], the torus-knot hierarchy and boundary-mode work from the Part I sequence [2, 3], the public balance law developed in the Part I capstone [4], the Part I transport tests [5], and the Life Number and tri-regime biology bridge from Part II [6]. The runtime papers therefore act as an execution layer: they keep the mathematics connected to commands, outputs, falsification tests, and domain-specific limits.

2 Why Ontology Is Needed

The core engine does not know what a nephron, a market, a mineral pore, or a photonic channel is. It only evolves state fields such as Φ , C , S , M_s , and Ψ_s . That ignorance is deliberate. A reusable runtime must not hard-code every domain into the kernel.

The ontology layer supplies meaning without contaminating the engine. It maps domain concepts to runtime variables, records relationships, and exposes those relationships to CDFL, the CLI, and the future web interface.

3 The Runtime Ontology Stack

3.1 Layer 0: Runtime Mechanics

The runtime layer is responsible for graph mechanics, state propagation, uncertainty flags, causal links, memory binding, regime tagging, and finite diagnostic summaries. It does not decide what a clinical or economic state means.

3.2 Layer 1: Core Meta-Ontology

The meta-ontology defines reusable categories:

- **Entity:** a node or bounded system.
- **Process:** a change pathway.
- **Flow:** a domain expression of Φ .
- **Constraint:** a domain expression of C .
- **Memory:** a domain expression of M_g .
- **Regime:** a qualitative label derived from runtime state.

3.3 Layer 2: Domain Ontologies

Domain adapters in `cdfd_runtime/domains` translate domain records into engine variables. The CLI currently reports 196 registered domain keys. The number matters less than the contract: each adapter must define how a domain maps to Φ and C , and must explain its interpretation limits.

3.4 Layer 3: Application Ontologies

Applications such as CDFD Studio, future scientific dashboards, or domain tools call the shared runtime backend rather than invent their own physics. This preserves one source of truth for finite checks, result envelopes, and CDFL execution.

3.5 Layer 4: Context and Memory

Context ontologies store user, project, dataset, or experiment-specific relationships. These guide interpretation, but they do not silently alter the physics kernel.

4 Schema Invention and Discovery

The local report `experiments/reports/NEW_DISCOVERIES.REPORT.md` describes a schema-invention demonstration in which a high- Ψ_s anomaly was assigned a new candidate label. The proper interpretation is not that the runtime has proven a new law of nature. The useful result is narrower: the discovery layer can detect an out-of-distribution regime, name it, record its features, and create a target for future falsification.

5 CDFL and Ontology Binding

CDFL binds ontology to execution. A `SET domain: physics` directive selects a semantic context; a `SYSTEM` block maps flow and constraint; a `RULE` block attaches an action to a threshold. The CLI now exposes this pipeline through:

```
python cdfd.py validate examples/heat_flow.cdf1
python cdfd.py run examples/heat_flow.cdf1
```

6 Falsification Conditions

An ontology mapping fails if:

- it cannot define measurable proxies for Φ and C ;
- its predicted regime thresholds do not reproduce under rerun;
- it adds no explanatory or predictive value beyond existing variables;
- it hides uncertainty behind authoritative labels.

7 Conclusion

The ontology is not decoration. It is the meaning infrastructure that lets one runtime serve many domains without pretending those domains are materially identical. Its strongest use is disciplined translation: map, run, report, falsify, and only then promote a candidate schema.

References

- [1] Steve Bico Mujjabi, MD. *Vortex Stability in a Constrained Transport Vacuum and the Origin of the Fine-Structure Constant*. CDFD Part I Paper I, preprint, 2026.
- [2] Steve Bico Mujjabi, MD. *The Universal Torus Knot Hierarchy: Z_n Symmetry, the Cinquefoil Family, and the General Structure of CDFT Particle Families*. CDFD Part I Paper VI, preprint, 2026.
- [3] Steve Bico Mujjabi, MD. *Even- n Torus Knots in CDFT: The Exclusion of $T(2,2)$, the Extension to $T(2,2m)$, and the Anti-Phase Mass Scaling Law*. CDFD Part I Paper IX, preprint, 2026.

- [4] Steve Bico Mujjabi, MD. *The Vacuum Equation of State: Deriving Torus Knot Self-Consistency from the Public CDFD Balance Law $\Psi = \Phi/C$* . CDFD Part I Paper X, preprint, 2026.
- [5] Steve Bico Mujjabi, MD. *Friction, Superconductivity, Plasma States, and Transport Mysteries: Observable Tests of Adaptive Constraint*. CDFD Part I Paper XII, preprint, 2026.
- [6] Steve Bico Mujjabi, MD. *CDFD Part II: Origins of Life and Tri-Regime Bioenergetics*. Public release archive, 2026, <https://doi.org/10.5281/zenodo.20250821>.